Interpolare polinomială

1. Vandermonde

Suport $S = [(x_0, y_0), (x_1, y_1) \dots (x_n, y_n)]$ – $n+1$ puncte cunoscute

$$\begin{cases} P_n(x_0) = y_0 \\ \vdots \\ P_n(x_n) = y_n \end{cases} \Rightarrow \begin{cases} a_0 + a_1 x_0 + a_2 x_0^2 + \dots + a_n x_0^n = y_0 \\ \vdots \\ a_0 + a_1 x_n + a_2 x_n^2 + \dots + a_n x_n^n = y_n \end{cases} \Rightarrow$$

$$\Rightarrow \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ \vdots & & & & \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$$A \cdot X = b$$

Concluzie: – n e de obicei mare $\Rightarrow$ A matrice plină mare $\Rightarrow$ nr. de condiționare mare $\Rightarrow$ instabilitate numerică

2. Lagrange

Definim $R(x) = (x - x_0)(x - x_1) \dots (x - x_n)$

Construim $\begin{cases} f_0(x) = R(x) / (x - x_0) \\ f_1(x) = R(x) / (x - x_1) \\ f_2(x) = R(x) / (x - x_2) \\ \vdots \\ f_n(x) = R(x) / (x - x_n) \end{cases}$

Obs: $f_0(x)$ e nenul pt x_0 și nul pt $x_1 \dots x_n$

$$P_n(x) = y_0 \cdot \frac{f_0(x)}{f_0(x_0)} + y_1 \cdot \frac{f_1(x)}{f_1(x_1)} + \dots + y_n \cdot \frac{f_n(x)}{f_n(x_n)}$$

Formalizare: $L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$ ← multiplicator Lagrange

$$\Rightarrow P_n(x) = \sum_{i=1}^{n} y_i \cdot L_i(x)$$

Concluzie:
- se poate aproxima eroarea
- instabil numeric
- oscilează în capete

3. Diferențe divizate (Newton)

$$f_n(x) = b_0 + b_1(x-x_0) + b_2(x-x_0)(x-x_1) + \dots + b_n(x-x_0)(x-x_1)\dots(x-x_n)$$

Liniare: $S = [(x_0, y_0), (x_1, y_1)]$

$f_1(x) = b_0 + b_1(x-x_0)$

$f_1(x_0) = b_0 + b_1(x_0-x_0) = \boxed{b_0 = y_0} = f(x_0)$ $\quad \boxed{b_0 = f[x_0]}$

$f_1(x_1) = b_0 + b_1(x_1-x_0) \Rightarrow b_1 = \dfrac{f(x_1)-f(x_0)}{x_1-x_0}$ $\quad \boxed{b_1 = f[x_0, x_1]}$

Pătratică: $S = [(x_0, y_0), (x_1, y_1), (x_2, y_2)]$

$f_2(x) = b_0 + b_1(x-x_0) + b_2(x-x_0)(x-x_1)$

$f_2(x_0) = \boxed{b_0 = f[x_0]}$

$f_2(x_1) = b_0 + b_1(x_1-x_0) \Rightarrow b_1 = \dfrac{f(x_1)-f(x_0)}{x_1-x_0} = f[x_0, x_1]$

$f_3(x_2) = b_0 + b_1(x_2-x_0) + b_2(x_2-x_0)(x_2-x_1)$

$$\Rightarrow b_2 = \frac{\dfrac{f(x_2)-f(x_1)}{x_2-x_1} - \dfrac{f(x_1)-f(x_0)}{x_1-x_0}}{x_2-x_0} = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2-x_0}$$

$$\Rightarrow b_2 = f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2-x_0}$$

$\| \; f[x_0, x_2]$

- $n+1$ puncte

$$f_n(x) = b_0 + b_1(x-x_0) + \dots + b_n(x-x_0)(x-x_1)\dots(x-x_n)$$

$b_0 = f[x_0]$, $b_1 = f[x_0, x_1]$ $b_2 = f[x_0, x_2] \dots b_n = f[x_0, x_n]$

$b_i = f[x_0, x_i]$ *

$$\boxed{f[x_i, x_j] = \frac{f[x_{i+1}, x_j] - f[x_i, x_{j-1}]}{x_j - x_i}}, \quad f[x_i, x_i] = f(x_i) = y_i$$

Polinomul de grad $j-i$ care trece prin $x_i, x_{i+1} \dots x_j$

Concluzie: - putem adăuga puncte noi ușor $\Rightarrow$ doar 1 coef. nou
- instabil numeric

4. **Neville** - nu ne interesează tot polinomul
- vrem aproximarea lui $f(x)$ pt. un anumit x
- construim polinoame de ord. mare folosind pol. de ord. mic

Aproximăm inițial $P_n(x)$ cu cele $n+1$ constante $y_0, \dots y_n$

Ex: $n=2$: $P_n(x) \sim P_{0,0}(x) = f(x_0) = y_0$

$\sim P_{1,1}(x) = f(x_1) = y_1$

$\sim P_{2,2}(x) = f(x_2) = y_2$

- combinăm câte 2 aprox. constante consec. într-o aprox. liniară

$$P_n(x) \sim P_{0,1}(x) = \frac{(x-x_1)\cdot P_{0,0}(x) - (x-x_0)P_{1,1}(x)}{x_0 - x_1}$$

$$\sim P_{1,2}(x) = \frac{(x-x_2)P_{1,1}(x) - (x-x_1)P_{2,2}(x)}{x_1 - x_2}$$

- combinăm câte 2 aprox liniare consec. într-o aprox. pătratică

$$P_n(x) \sim P_{0,2}(x) = \frac{(x-x_2)\cdot P_{0,1}(x) - (x-x_0)\cdot P_{1,2}(x)}{x_0 - x_2}$$

$P_{0,0} = f(x_0)$ → $P_{0,1}(x)$

$P_{1,1} = f(x_1)$ → $P_{0,1}(x)$, $P_{1,2}(x)$

$P_{2,2} = f(x_2)$ → $P_{1,2}(x)$, $P_{2,3}(x)$

$P_{3,3} = f(x_3)$ → $P_{2,3}(x)$

$P_{0,1}(x)$, $P_{1,2}(x)$ → $P_{0,2}(x)$

$P_{1,2}(x)$, $P_{2,3}(x)$ → $P_{1,3}(x)$

$P_{0,2}(x)$, $P_{1,3}(x)$ → $P_{0,3}(x)$

Algoritm :

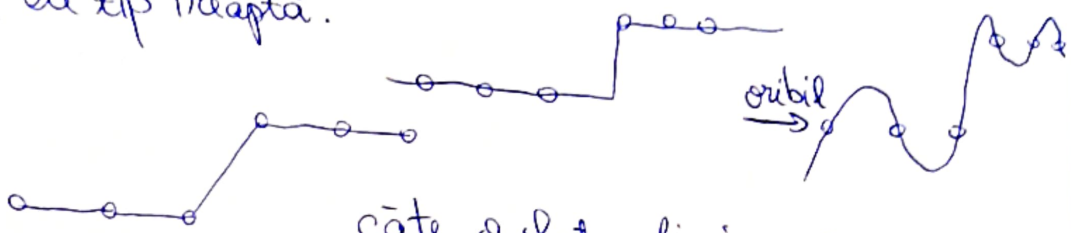
$$P_{i,j}(x) = \frac{(x-x_j)\cdot P_{i,j-1}(x) - (x-x_i)\cdot P_{i+1,j}(x)}{x_i - x_j}$$

$P_{i,i}(x) = f(x_i)$

$P_{0,n}$ - pol. căutat

(5) Spline

Funcție originală de tip treaptă.



Spline liniar ↘

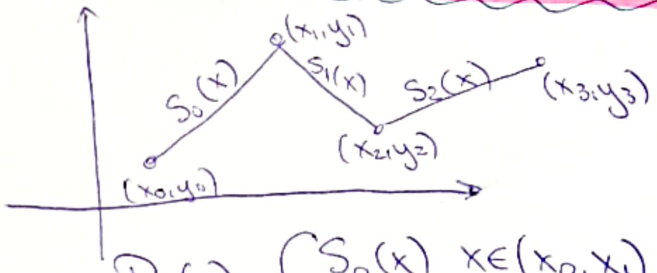
cate o fct. liniară între oricare 2 pct. consec.

Spline cubic →



folosind fct. de îmbinare de grad 3

6.1) Spline cu fct de îmbinare liniare



$$\begin{cases} \frac{y_2-y_1}{x_2-x_1} = m \\ y-y_1 = m(x-x_1) \end{cases} \leftarrow S_1$$

$$P_n(x) = \begin{cases} S_0(x) & x \in (x_0, x_1) \\ S_1(x) & x \in (x_1, x_2) \\ S_2(x) & x \in (x_2, x_3) \end{cases}$$

6.2) Spline cu fct. de îmbinare cubice (grad 3 = permite cel puțin 1 inflexiune)

$$S = [(x_0,y_0), (x_1,y_1) \dots (x_n,y_n)]$$

$$S_i : [x_i, x_{i+1}) \to R \quad , S_i(x) = a_i + b_i(x-x_i) + c_i(x-x_i)^2 + d_i(x-x_i)^3$$

n fct. spline · 4 coef ⇒ 4n necunoscute

I. funcții de clasă C^2 (deriv. de 2 ori + derivate cont.)

I.1) fiecare spline trece prin punctul de control:

$$S_i(x_i) = f(x_i) = y_i \quad i = \overline{0; n-1}$$

$$S_{n-1}(x_n) = f(x_n) = y_n \quad \Rightarrow \underline{n+1} \text{ ec.}$$

I.2) racordăm funcțiile (unde se termină S_i începe S_{i+1}) ⇒ asigurăm continuitate

$$S_i(x_{i+1}) = S_{i+1}(x_{i+1}) \quad i = \overline{0:n-2} \quad \Rightarrow \underline{n-1} \text{ ec.}$$

I 3) funcțiile au aceeași pantă în pct. de contact ⇒ asigurăm continuitate C^1

$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}) \quad i = \overline{0 : n-2}$ — $n-1$ ec.

I 4) funcțiile au aceeași convexitate în pct. de contact ⇒ asigurăm continuitate C^2

$S_i''(x_{i+1}) = S_{i+1}''(x_{i+1}) \quad i = \overline{0 : n-2}$ — $n-1$ ec.

I 5) 2 ecuații:

Spline naturale	Spline tensionat
$S_0''(x_0) = 0$	$S_0''(x_0) = f'(x_0)$
$S_n''(x_0) = 0$	$S_n''(x_n) = f'(x_n)$

Tensionat – oferă aprox. mai bună, dar necesită cunoașterea $f'(x_0)$ și $f'(x_n)$

II. Funcții de clasă C^1 – necesită cunoașterea derivatei

$S = [(x_0, y_0, y_0'), (x_1, y_1, y_1') \dots (x_n, y_n, y_n')$

II 1) $S_i(x_i) = f(x_i) \quad i = 0 : n-1$

$S_{n-1}(x_n) = f(x_n)$

$S_i'(x_i) = f'(x_i) = y_i' \quad i = 0 : n-1$

$S_{n-1}'(x_n) = f'(x_n) = y_n'$

II 2) $S_i(x_{i+1}) = S_{i+1}(x_{i+1}) \quad i = 0 : n-2$

$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}) \quad i = 0 : n-2$

Ex: $S = [(1,5), (3,10), (4,-10)] \quad n = 2 \Rightarrow$ Spline C^2

($S_0(x)$ pe $(1,5),(3,10)$; $S_1(x)$ pe $(3,10),(4,-10)$)

$S_0(x) = a_0 + b_0(x-x_0) + c_0(x-x_0)^2 + d_0(x-x_0)^3$

$S_1(x) = a_1 + b_1(x-x_1) + c_1(x-x_1)^2 + d_1(x-x_1)^3$

$S_0'(x) = b_0 + 2c_0(x-x_0) + 3d_0(x-x_0)^2$

$S_1'(x) = b_1 + 2c_1(x-x_1) + 3d_1(x-x_1)^2$

$S_0''(x) = 2c_0 + 6d_0(x-x_0)$

$S_1''(x) = 2c_1 + 6d_1(x-x_1)$

1) $S_0(x_0) = y_0 \Rightarrow \underline{a_0 = 5}$

$S_1(x_1) = y_1 \Rightarrow \underline{a_1 = 10}$

$S_1(x_2) = y_2 \Rightarrow \underbrace{a_1}_{10} + b_1(\underbrace{x_2 - x_1}_{4-3=1}) + c_1(x_2 - x_1)^2 + d_1(x_2 - x_1)^3 = -10$

$\Rightarrow \underline{a_1 + b_1 + c_1 + d_1 = -10}$

2) $S_0(x_1) = S_1(x_1) \Rightarrow \underbrace{a_0}_{5} + b_0(\underbrace{x_1 - x_0}_{3-1=2}) + c_0(x_1 - x_0)^2 + d_0(x_1 - x_0)^3 = \underbrace{a_1}_{10}$

$\Rightarrow \underline{a_0 + 2b_0 + 4c_0 + 8d_0 - a_1 = 0}$

3) $S_0'(x_1) = S_1'(x_1) \Rightarrow b_0 + 2c_0(\underbrace{x_1 - x_0}_{3-1=2}) + 3d_0(x_1 - x_0)^2 = b_1$

$\Rightarrow \underline{b_0 + 4c_0 + 12d_0 - b_1 = 0}$

4) $S_0''(x_1) = S_1''(x_1) \Rightarrow 2c_0 + 6d_0(\underbrace{x_1 - x_0}_{2}) = 2c_1 \Rightarrow \underline{c_0 + 6d_0 - c_1 = 0}$

5) Natural: $S_0''(x_0) = 0 \Rightarrow 2c_0 = 0 \Rightarrow \underline{c_0 = 0}$

$S_1''(x_2) = 0 \Rightarrow 2c_1 + 6d_1(x_2 - x_1) = 0 \Rightarrow 2c_1 + 6d_1 = 0 \Rightarrow \underline{c_1 + 3d_1 = 0}$

a_0	b_0	c_0	d_0	a_1	b_1	c_1	d_1			
1	0	0	0	0	0	0	0	a_0	=	5
0	0	0	0	1	0	0	0	b_0		10
0	0	0	0	1	1	1	1	c_0		-10
1	2	4	8	-1	0	0	0	d_0		0
0	1	4	12	0	-1	0	0	a_1		0
0	0	1	6	0	0	-1	0	b_1		0
0	0	1	0	0	0	0	0	c_1		0
0	0	0	0	0	0	1	3	d_1		0

$\Rightarrow \begin{cases} S_0(x) = 5 + 10(x-1) - 1{,}875(x-1)^3 \\ S_1(x) = 10 - 12{,}5(x-3) - 11{,}25(x-3)^2 + 3{,}75(x-3)^3 \end{cases}$

$P_n(x) = \begin{cases} S_0(x) & x \in [1,3) \\ S_1(x) & x \in [3,4) \end{cases}$

(6) Curbe Bezier

$S=[(x_0,y_0),(x_1,y_1)]$ polinoame Bernstein: $B_{01}(t)=(1-t)P_0+tP_1$ $0\le t\le 1$

$B_{02}(t)=(1-t)B_{01}(t)+tB_{12}(t)$ $\left(B_{01}(t)=(1-t)P_0+tP_1,\ B_{12}(t)=(1-t)P_1+tP_2\right)$

Algoritm de Casteljau

$$B_{0,n}(t)=(1-t)B_{0,n-1}(t)+tB_{1,n}(t) \quad t\in[0,1]$$

$$B_{i,i}(t)=P_i$$

→ Curba Bezier printre punctele $[P_0..P_n]$ evaluată la mom. t

(7) Interpolare Fourier

1) Compunerea și descompunerea undelor

Căutăm $2N$ sinusoidale care aproximează funcția

2) Transformata Fourier Discretă

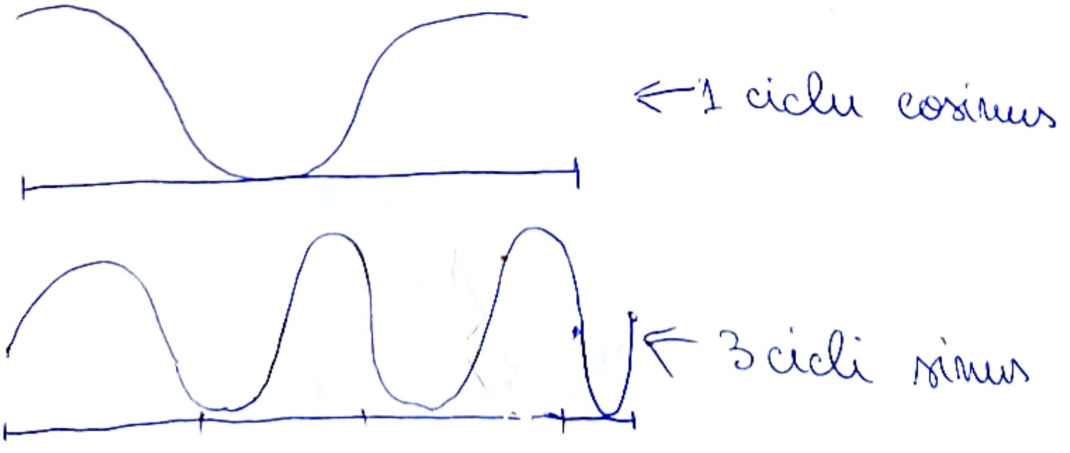
Vrem să codificăm $f(x)=2\cos\left(\frac{\pi}{4}x\right)$ pe $[0,8]$

Cunoaștem doar 8 sample-uri din fct.: $(0,f(0)),(1,f(1))\dots(7,f(7))$

Vrem să vedem cât de mult seamănă punctele pe care le cunoaștem cu:

1) punctele de pe o cosinusoidă cu un ciclu complet pe întreg domeniul;
2) sinusoidă
3) cu 2 cicli (2Hz) pe
6)
7) 7 Hz

Ex:



Cum putem măsura similaritatea dintre 2 fct, într-un set de pct. ?

$$corr(f_1, f_2, X) = \sum_{i=1}^{n} f_1(x_i) \cdot f_2(x_i)$$

$\{x_1 \dots x_n\}$

$f(x) = 2\cos\left(\frac{\pi}{4}x\right)$

$S = [(0, f(0)) \dots (7, f(7))]$ $\xrightarrow{DFT}$

	corr cos	corr sin
K=0	0	0i
K=1	8	0i
K=2	0	0i
K=3		
K=4		
K=5		
K=6		
K=7	8	0i

DFT formal:

$$X_K = \sum_{n=0}^{N-1} x_n e^{-\frac{i \cdot 2\pi}{N} \cdot Kn} = \sum_{n=0}^{N-1} x_n \cdot \left[\cos\left(\frac{2\pi}{N} Kn\right) - i \sin\left(\frac{2\pi}{N} Kn\right)\right]$$

Am codificat o undă într-o formă discretă.

Pt. a decodifica facem IDFT $\Rightarrow \{x_n\}$ + interpolare $\simeq$ fct. inițială din care am fct. sample

Cum transformăm în algebră liniară?

1) $\mathbb{R} \to \mathbb{C}$

2) $z \in \mathbb{R} \Rightarrow$ lungimea vect $= \sqrt{z^T \cdot z}$

$z \in \mathbb{C} \Rightarrow$ ——— $\sqrt{\bar{z}^T \cdot z}$, $\bar{z}^T = z^H$ (Hermitian z)

3) prod scalar $x, y \to \bar{y}^T x = y^H \cdot x$

4) A simetrică în $\mathbb{C} \Leftrightarrow \bar{A}^T = A \Leftrightarrow A^H = A$

5) Q - ortogonală dacă $Q^H \cdot Q = I$ (unitară)

DFT sub formă matricială: $F \cdot X = x$ $\xrightarrow{x_0, x_1 \dots x_{n-1}}$ samples din repr. continuă

$X[0], \dots X[N-1]$

$$F = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & w & w^2 & & w^{n-1} \\ 1 & w^2 & w^4 & & w^{2(n-1)} \\ \vdots & & \vdots & & \vdots \\ 1 & w^{n-1} & w^{2(n-1)} & & w^{(n-1)^2} \end{bmatrix}, \quad w = e^{\frac{2\pi i}{N}}$$

$X = F^H * \cdot [1 \; w \; w^2 \dots w^{N-1}]^T$ → un ciclu complet cos și sin

$[1 \; w^2 \; w^4 \dots w^{2(N-1)}]^T$ → 2 cicluri complete cos și sin

Ce înseamnă FFT?

$$F_N = \begin{bmatrix} I_M & D_M \\ I_M & -D_M \end{bmatrix} \begin{bmatrix} F_M & 0 \\ 0 & F_M \end{bmatrix} \begin{bmatrix} 1 & & & & & \\ & & 1 & & & \\ & 1 & & & 1 & \\ & & & 1 & & \\ & & & & & 1 \end{bmatrix}$$

→ matrice de permutare par-impar

$$M = \tfrac{1}{2}N$$

$$D_M = \begin{bmatrix} 1 & & & \\ & \omega_N & & \\ & & \ddots & \\ & & & \omega_N^{(n-1)} \end{bmatrix}$$

F·X ia n^2 înmulțiri pt DFT

F·X ia $\frac{1}{2} n \log n$ înmulțiri pt. FFT deoarece matricele derivate din ea în ce mai rar

---

Derivare și integrare numerică

⊛ Estimarea erorii interpolării polinomiale

Fie $f(x)$ cu $x \in [a,b]$ și un suport al interpolării $S = [(x_0, f(x_0)), \dots, (x_n, f(x_n))]$. Fie $P_n(x)$ polinomul de grad n ce aproximează $f(x)$ pe $[a,b]$, unde $P_n(x_i) = f(x_i)$

$\text{eroare}(x) = f(x) - P_n(x) \quad x \in [a,b]$

Teoremă: există o val $\xi \in [a,b]$ aî. $e(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) \cdot \prod_{i=0}^{n} (x - x_i)$, $\forall x \in [a,b]$

Integrare numerică

Vrem să găsim o aproximare $I(f) = \int_a^b f(x)\,dx$.

→ împarte $[a,b]$ în subintervale

→ aproximează fiecare subinterval cu un polinom $P_i(x) \simeq f(x)$

→ integrează $P_i(x)$ și le adună

$P_i(x)$ – e ales simplu → pb simplă, rapidă, inexactă

e ales complex → pb complexă, lentă, exactă

Regula trapezelor

→ împarte $[a,b]$ în n subintervale $x_0 = a$, $x_i < x_{i+1}$, $x_n = b$

→ pe $[x_i, x_{i+1}]$ aproximăm $f(x)$ cu polinomul P_i cu interpolare liniară

$$\int_{x_i}^{x_{i+1}} f(x)\,dx \approx \int_{x_i}^{x_{i+1}} P_i(x)\,dx = \frac{1}{2}\left(f(x_i)+f(x_{i+1})\right)\cdot(x_{i+1}-x_i)$$

$$\int_a^b f(x)\,dx = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} p_i(x)\,dx$$

Newton-Cotes (puncte echidistante)

$$h = \frac{b-a}{n} \qquad x_{i+1} - x_i = h$$

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n-1} \frac{h}{2}\left(f(x_i)+f(x_{i+1})\right) = \frac{h}{2}\left[f(x_0)+f(x_n)+2\sum_{i=1}^{n-1} f(x_i)\right]$$

$$\overset{\text{Not.}}{=} T(f,h)$$

Octave:

```
function [T] = trapezoidal (f,a,b,n)
  h=(b-a)/n;
  X=a+h :h: b-h
  T= h*((f(a)+f(b))/2 + sum(f(X)));
endfunction
```

Analiza erorii

$$E_T(f,h) = I(f) - T(f,h) = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x) - p_i(x)\,dx$$

$$E_T(f,h) = \sum_{i=0}^{n-1} E_{T,i}(f,h), \text{ unde } E_{T,i}(f,h) = \int_{x_i}^{x_{i+1}} f(x) - p_i(x)\,dx$$

$$f(x) - p_i(x) = \frac{1}{2} f''(\xi_i)(x-x_i)(x-x_{i+1}), \quad \xi_i \in (x_i, x_{i+1})$$

$$E_{T,i}(f,h) = \frac{1}{2} f''(\xi_i) \int_{x_i}^{x_{i+1}} (x-x_i)(x-x_{i+1})\,dx = -\frac{1}{12} h^3 f''(\xi_i)$$

$$E_T(f,h) = \sum_{i=0}^{n-1} -\frac{h^3}{12} f''(\xi_i) = -\frac{h^3}{12}\left[\sum_{i=0}^{n-1} f''(\xi_i)\right]\cdot\frac{1}{n}\cdot\frac{b-a}{h}$$

$$= \frac{(a-b)h^2}{12}\cdot f''(\xi)$$

($f''(\xi)$, $\xi \in [a,b]$; $\frac{1}{n}\cdot\frac{b-a}{h} = 1$)

$$|E_T(f,h)| \le \frac{b-a}{12} h^{(2)} \cdot \max_{x\in[a,b]} |f''(x)| \Rightarrow \text{metodă de ordin 2}$$

Regula Simpson

→ împărțim $[a,b]$ în $2n$ subintervale $x_0=a$ $x_n=b$ $h=\frac{b-a}{2n}$

→ pe $[x_{2i}, x_{2i+2}]$ aproximăm $f(x)$ cu un polinom p_i

3 pct. ⇒ interpolare pătratică

$p_i(x_{2i})=f(x_{2i})$; $p_i(x_{2i+1})=f(x_{2i+1})$; $p_i(x_{2i+2})=f(x_{2i+2})$

(fac $p_i(x)$ cu Lagrange de ord. 2)

$$p_i(x)=f(x_{2i})\cdot L_{2i}(x)+f(x_{2i+1})\cdot L_{2i+1}(x)+f(x_{2i+2})\cdot L_{2i+2}(x)$$

$$L_{2i}(x)=\frac{(x-x_{2i+1})(x-x_{2i+2})}{(x_{2i}-x_{2i+1})(x_{2i}-x_{2i+2})}$$

$[x_{2i}, x_{2i+1}, x_{2i+2}]$

$$p_i(x)=\frac{1}{2h^2}f(x_{2i})(x-x_{2i+1})(x-x_{2i+2})-\frac{1}{h^2}f(x_{2i+1})(x-x_{2i})(x-x_{2i+2})+\frac{1}{2h^2}f(x_{2i+2})(x-x_{2i})(x-x_{2i+1})$$

$$I=\int_{x_{2i}}^{x_{2i+2}} p_i(x)=\frac{2}{3}h^3+\frac{4}{3}h^3+\frac{2}{3}h^3=\frac{8}{3}h^3$$

$$=\frac{1}{2h^2}f(x_{2i})\cdot\frac{2}{3}h^3+\frac{1}{h^2}\cdot f(x_{2i+1})\cdot\frac{4}{3}h^3+\frac{1}{2h^2}f(x_{2i+2})\frac{2}{3}h^3=$$

$$=\frac{h}{3}\left(f(x_{2i})+4f(x_{2i+1})+f(x_{2i+2})\right)$$

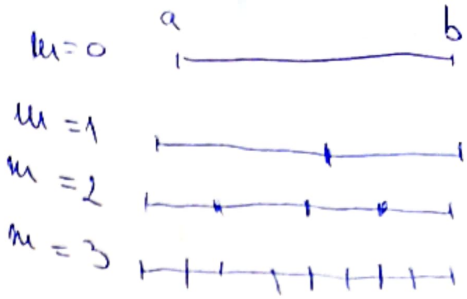
Regula Simpson: $\mathcal{I}_i=\frac{h}{3}\left(f(x_{2i})+4f(x_{2i+1})+f(x_{2i+2})\right)$

$$\Rightarrow S(f,h)=\frac{h}{3}\left(f(x_0)+f(x_{2n})+4\sum_{i=1}^{n}f(x_{2i-1})+2\sum_{i=1}^{n-1}f(x_{2i})\right)$$

Analiza erorii: $E_{S,i}(f,h)=-\frac{1}{90}h^5 f^{(4)}(\xi_i)$ $\xi_i\in(x_{2i},x_{2i+2})$

$E_S(f,h)=\frac{b-a}{180}h^4\cdot f^{(4)}(\xi)$, $\xi\in(a,b)$ → metodă de ord. 4.

Reguli recursive



$h_m = \frac{b-a}{2^m}$, $h_{m+1} = \frac{1}{2} h_m$; $n = 2^m$

$T(f, h) = h\left[\frac{1}{2}f(x_0) + \frac{1}{2}f(x_n) + \sum_{i=1}^{n-1} f(x_i)\right]$

$T(f, h_m) = h_m\left[\frac{1}{2}f(a) + \frac{1}{2}f(b) + \sum_{i=1}^{2^m-1} f(a+i\cdot h_m)\right]$

$T(f, h_{m+1}) = h_{m+1}\left[\frac{1}{2}f(a) + \frac{1}{2}f(b) + \sum_{i=1}^{2^{m+1}-1} f(a+i\cdot h_m)\right] =$

$= \frac{h_m}{2}\left[\frac{1}{2}f(a) + \frac{1}{2}f(b) + \sum_{i=1}^{2^m-1} f(a+i h_m) + \sum_{i=0}^{2^m-1} f(a+(2i+1)\cdot h_{m+1})\right]$

pct. de la pas m — pct. noi adăugate

$= \frac{1}{2}T(f, h_m) + \frac{h_m}{2}\sum_{i=0}^{2^m-1} f(a+(2i+1)h_{m+1})$

Dacă aproximarea pt. m nu e destul de bună, poți trece la m+1 -> flexibilitate
+nu e nev. să recalculezi în 1/2 din pct.

Extrapolare Richardson

Având $T(f, h)$, $T(f, \frac{h}{2})$, $T(f, \frac{h}{4})$... o secv. de aprox. cu regula trapezelor cu diferite val. ale lui h => le putem combina pt. a det. aproximări de ordin mai mare.

Dacă $f^{(n)}$ există și e mărg. -> eroarea pt regula trapezelor:

① $E(f, h) = a_2 h^2 + a_4 h^4 + a_6 h^6 + \dots + a_n h^n$

② $E(f, \frac{h}{2}) = a_2 \left(\frac{h}{2}\right)^2 + a_4 \left(\frac{h}{2}\right)^4 + \dots + a_n \left(\frac{h}{2}\right)^n$ -> ord. 2

$2^2 \cdot ② - 1 \cdot ① \Rightarrow 3I(f) = 4T(f, \frac{h}{2}) - T(f, h) + a_4' h^4 + a_6' h^6 + \dots$

$\Rightarrow I(f) = \frac{4}{3}T(f, \frac{h}{2}) - \frac{1}{3}T(f, h) + \tilde{a}_4 h^4 + \tilde{a}_6 h^6 + \dots$ -> ord. 4

$$U(h) = \frac{4}{3}T(f, \frac{h}{2}) - \frac{1}{3}T(f, h)$$

Integrare Romberg - iterăm ideea de la Richardson

① $I(f) = U(h) + \tilde{a}_4 h^4 + \tilde{a}_6 h^6 + \dots$

② $I(f) = U\left(\frac{h}{2}\right) + \tilde{a}_4 \left(\frac{h}{2}\right)^4 + \tilde{a}_6 \left(\frac{h}{2}\right)^6 + \dots$

$2^4 \cdot ② - 1 \cdot ① \Rightarrow 15 I(f) = 2^4 U(h/2) - U(h) + a_6' h^6 + \dots$

$I(f) = V(h) + \tilde{a}_6 h^6 + \dots \rightarrow$ ord. 6

$$V(h) = \frac{2^4 U(h/2) - U(h)}{(2^4 - 1)}$$

Alg:

$H = b - a$

$R(0,0) = T(f, H) = \frac{H}{2}(f(a) + f(b))$

$R(1,0) = T(f, \frac{H}{2}) = \frac{H}{4}(f(a) + f(b))$

$R(2,0) = T(f, \frac{H}{4}) = \frac{H}{8}(f(a) + f(b))$

$R(n,0) = T(f, \frac{H}{2^n}) = \frac{H}{2^{n+1}}(f(a) + f(b))$

Triunghi Romberg:

$R(0,0) \rightarrow R(1,1) \rightarrow R(2,2) \dots R(n,n)$ ← $U(h)$, ↓ $V(h)$

$R(1,0) \rightarrow R(2,1) \dots$

$R(2,0) \dots$

$R(n,0) \rightarrow R(n,1) \rightarrow R(n,2)$

ord. 2 ord. 4 ord. 6 ord. $2(n+1)$

$$R(n,m) = R(n,m-1) + \frac{R(n,m-1) - R(n-1,m-1)}{2^{2m} - 1}$$

$I(f) = R(n,m) + \theta(h^{2(m+1)}) \rightarrow$ ord. $2(m+1)$

Cuadraturi adaptive

- folosim intervale adaptive $\Rightarrow h$ mai mic în regiunile unde f variază mai mult
- ex. folosind Simpson:

$[a,b]$, $h = \frac{b-a}{2} \Rightarrow S_1[a,b] = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$

$E_1[a,b] = -\frac{1}{90} h^5 f^{(4)}(\xi)$

$I_f[a,b] = S_1[a,b] + E_1[a,b]$

- împărțim $[a,b]$ în 2, $c = \frac{a+b}{2}$

$$I(f)[a,b] = I(f)[a,c] + I(f)[c,b] = S_1[a,c] + E_1[a,c] + S_1[c,b] + E_1[c,b] =$$

$$= S_2[a,b] + E_2[a,b], \quad S_2[a,b] = S_1[a,c] + S_1[c,b]$$

$$E_2[a,b] = E_2[a,c] + E_1[c,b] =$$

$$= -\frac{1}{90}\left(\frac{h}{2}\right)^5 \left(f^{(4)}(\xi_1) + f^{(4)}(\xi_2)\right) \simeq \frac{1}{2^4} E_1[a,b]$$

① $I(f) = S_1 + E_1$ ② $I(f) \in S_2 + \frac{1}{2^4} E_1$ ② - ① → $S_2 - S_1 = (2^4 - 1) E_2$

Putem aproxima $E_2 = \frac{1}{15}(S_2 - S_1)$

Când $f^{(4)}$ nu variază mult, putem aproxima $I = S_2 + \frac{S_2 - S_1}{15}$ în loc de $I = S_2$

Algoritm

```
function [S] = adaptive_simpson(f, a, b, tol)
    S1 = simpson(a,b)
    S2 = simpson(a,c) + simpson(c,b)
    dacă |S2 - S1| < 15·tol   (dacă f^(4) nu variază)
        S = S2 + (S2 - S1)/15
    altfel
        c = (a+b)/2
        left = adaptive_simpson(f, a, c, tol/2)
        right = adaptive_simpson(f, c, b, tol/2)
        S = left + right
endfunction
```

Concluzii: când $f^{(4)}$ nu e const/liniar → eroarea aprox. e mare → micșorăm intervalul recursiv până când $f^{(4)}$ nu variază (asta pt Simpson)

quad(@f, a, b)

Cuadraturi Gaussiene

Toate integralele numerice arată astfel: $\int_a^b f(x)dx \simeq A_0 f(x_0) + A_1 f(x_1) + \dots + A_m f(x_m)$

$x_i \in [a,b]$ = noduri

A_i = greutăți / coeficienți

Reg. trapezului: $x_0 = a$, $x_1 = b$, $A_0 = A_1 = \frac{b-a}{2}$

Reg. Simpson: $x_0 = a$, $x_1 = \frac{a+b}{2}$, $A_0 = A_2 = \frac{1}{6}(b-a)$, $A_1 = \frac{2}{3}(b-a)$

Întâi fixăm nodurile, apoi ajustăm coef.

Acum alegem x_i, A_i $(i = 0, \dots N)$ ai, algoritmul ne oferă val. exactă a integralei $\int_a^b f(x)dx = A_0 f(x_0) + \dots + A_m f(x_m)$, dacă $f \in \mathcal{P}_m$ pt. fct. polinomiale de grad max m

Exemplu:

- fixăm $[-1,1]$

$N=1 \Rightarrow x_0, x_1$ și A_0, A_1

$\int_{-1}^{1} f(x)dx = A_0 f(x_0) + A_1 f(x_1)$ $f \in \mathcal{P}^m$, regula trb să fie exactă pt $1, x, \dots, x^m$

4 nec. $(x_0, x_1, A_0, A_1) \Rightarrow m = 3 = 2N+1$ grad de valabilitate (grad max până la care un polinom e aproximat exact)

$f(x) = 1 \Rightarrow A_0 + A_1 = \int_{-1}^{1} 1\,dx = 2$

$f(x) = x \Rightarrow A_0 x_0 + A_1 x_1 = \int_{-1}^{1} x\,dx = 0$

$f(x) = x^2 \Rightarrow A_0 x_0^2 + A_1 x_1^2 = \int_{-1}^{1} x^2 dx = 2/3$ $\Rightarrow x_0 = -\frac{1}{\sqrt{3}}$, $x_1 = \frac{1}{\sqrt{3}}$, $A_0 = 1$, $A_1 = 1$

$f(x) = x^3 \Rightarrow A_0 x_0^3 + A_1 x_1^3 = \int_{-1}^{1} x^3 dx = 0$

simetrie

$\Rightarrow \int_{-1}^{1} f(x)dx \simeq f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$ = pt. $f \in \mathcal{P}^3$

Interval diferit de $[-1,1]$, $[a,b]$

$t = \frac{2x-(a+b)}{b-a}$, $x = 1/2(b-a)t + 1/2(b+a)$

$-1 \le t \le 1 \Rightarrow a \le x \le b$

Ex: $\int_0^{10} f(x)dx = \int_{-1}^{1} f\left((1/2)(10-0)\cdot t + 1/2(10+0)\right) 5\,dt = 5\int_{-1}^{1} f(5t+5)dt$

$f(x) = 7x^3 - 8x^2 - 3x + 3$

$\int_0^{10} f(x)dx = 14713$

$5\int_{-1}^{1} f(5t+5)dt = 5\left(f\left(5\cdot\frac{-1}{\sqrt{3}}+5\right) + f\left(5\cdot\frac{1}{\sqrt{3}}+5\right)\right) = 14713$

Derivare numerică

$\begin{cases} x' = f(t,x) & \text{- ec. dif.} \\ x(t_0) = x_0 & \text{- cond. init.} \end{cases}$

h = step

$t_{k+1} = t_k + h$; $t_k = t_0 + k \cdot h$ $\quad t_0 < t_1 < \dots < t_n$

având o ec. dif. și $t_n \Rightarrow$ aflăm $x_n \simeq x(t_n)$ $\quad n = 1, 2 \dots N$

Metode bazate pe serii Taylor

Fie $t_1 = t_0 + h \Rightarrow$ aproximăm $x_1 \sim x(t_1) = x(t_0 + h)$

Dezv. Taylor a lui $x(t_0+h)$ în t_0:

$$x(t_0+h) = x(t_0) + hx'(t_0) + \frac{1}{2}h^2 x''(t_0) + \dots = \sum_{m=0}^{\infty} \frac{1}{m!} h^m \cdot x^{(m)}(t_0)$$

Luăm doar primii $m+1$ termeni

Eroarea la fiecare pas: $x(t_0+h) - x_1 = \frac{1}{(m+1)!} h^{(m+1)} \cdot x^{(m+1)}(\xi)$, $\xi \in (t_0, t_1)$ → met. de ord. $m+1$

- $m = 1 \Rightarrow x_1 = x_0 + hx'(t_0) = x_0 + h \cdot f(t_0, x_0)$ ($\hookrightarrow x(t_0)$) $\equiv$ Forward Euler Step

$$x_{k+1} = x_k + h f(x_k, t_k) \quad k = 0, \dots N-1$$

- $m = 2 \Rightarrow x_1 = x_0 + hx'(t_0) + \frac{h^2}{2} x''(t_0)$

$$x'' = \frac{d}{dt} x'(t) = \frac{d}{dt} f(t, x(t)) = f_t + f_x \cdot x' = f_t + f_x \cdot f$$

$$\Rightarrow x_1 = x_0 + hx'(t_0) + \frac{h^2}{2}\left[f_t(t_0,x_0) + f_x(t_0,x_0) \cdot f(t_0,x_0)\right]$$

($\hookrightarrow$ derivata $\frac{\partial f}{\partial t}$, $\frac{\partial f}{\partial x}$)

$$x_{k+1} = x_k + hf(x_k,t_k) + \frac{h^2}{2}\left[f_t(t_k,x_k) + f_x(t_k,x_k)\cdot f(t_k,x_k)\right], \quad k = 0, \dots N-1$$

Runge - Kutta

Folosim doar $f(t,x)$, nu și derivatele ei

Met. de ord. I : Euler

Met. de ord. II : $h = t_{k+1} - t_k$; $x_{k+1} = x_k + \frac{1}{2}(K_1 + K_2)$, unde: — Heun / RK2

$K_1 = hf(t_k, x_k) \equiv$ Euler

$K_2 = hf(t_k + h, x_k + K_1) \equiv$ Euler pt t_{k+1}

$x_{k+1} = x_k + hf(t_k, x_k)$ (t_{k+1}, x_{k+1})

Heun văzută ca met. trapezelor:

$x(t_k+h) = x(t_k) + \int_{t_k}^{t_k+h} x'(t)\,dt$

$K_1 \simeq h x'(t_k)$; $K_2 \simeq h x'(t_k+h)$

Regula trap pe $(t_k, x'(t_k))$ și $(t_k+h, x'(t_k+h))$:

$\int_{t_k}^{t_k+h} x'(t)\,dt \simeq \frac{h}{2}\left[\underbrace{x'(t_k)}_{\frac{K_1}{h}} + \underbrace{x'(t_k+h)}_{\frac{K_2}{h}}\right] = \frac{1}{2}(K_1+K_2)$

• Met. de ord. 4 RK4 Clasic:

$x_{k+1} = x_k + \frac{1}{6}[K_1+2K_2+2K_3+K_4]$, unde:

$$\begin{cases} K_1 = h f(t_k, x_k) \\ K_2 = h f(t_k + 1/2 h, x_k + 1/2 K_1) \\ K_3 = h f(t_k + 1/2 h, x_k + 1/2 K_2) \\ K_4 = h f(t_k + h, x_k + K_3) \end{cases}$$

Metode Runge-Kutta adaptive

Alg:

1) Calc. $x(t+h)$ din $x(t)$ cu met. 1 $= x(t+h)$

2) ——————— met 2 $= \bar{x}(t+h)$

3) $|x(t+h) - \bar{x}(t+h)| \equiv$ eroare locală

4) err $\gg$ tol $\Rightarrow h = h/2$

5) err $\ll$ tol $\Rightarrow h = 2^* h$

6) err $\simeq$ tol $\Rightarrow$ h nemodificat

Metode explicite (Adam - Bashforth)

$x(t_{n+1}) = x(t_n) + \int_{t_n}^{t_{n+1}} x'(t)\,dt$

Aproximăm f printr-o interpolare polinomială având $S = [(t_{n-k}, f(t_{n-k}, x_{n-k})), (t_{n-k+1}, f(t_{n-k+1}, x_{n-k+1})), \ldots (t_n, f(t_n, x_n))] = [(t_{n-k}, f_{n-k}), (t_{n-k+1}, f_{n-k+1}) \ldots, (t_n, f_n)]$

folosim ultimele $k+1$ pct.

-găsim pol $P_K(t)$ (grad K) care interpolează $S \Rightarrow$ Lagrange: $P_K(t) = f_n L_n(t) +$

$+ f_{n-1} L_{n-1}(t) + \ldots + f_{n-K} L_{n-K}(t) \Rightarrow x_{n+1} = x_n + \int_{t_n}^{t_{n+1}} P_K(s)\,ds$

$x_{n+1} = x_n + \int_{t_n}^{t_n+h} (f_n L_n(t) + f_{n-1} L_{n-1}(t) + \ldots + f_{n-K} L_{n-K}(t)) =$

$= x_n + f_n \int_{t_n}^{t_n+h} L_n(t)\,dt + \ldots + f_{n-K} \int_{t_n}^{t_n+h} L_{n-K}(t)\,dt$

$\Rightarrow x_{n+1} = x_n + h(b_0 f_n + b_1 f_{n-1} + \ldots + b_K f_{n-K})$

$b_K = \frac{1}{h} \int_{t_n}^{t_n+h} L_{n-K}(t)\,dt$

Ex: $K=0 \rightarrow$ folosim doar $(t_n, f_n) \Rightarrow P_0(t) = f_n$

$\rightarrow x_{n+1} = x_n + h f(t_n, x_n)$ = Euler

$K=1 \rightarrow (t_{n-1}, f_{n-1}), (t_n, f_n)$

$x'(s) \approx P_1(s) = f_{n-1} + \frac{f_n - f_{n-1}}{h}(s - t_{n-1})$

$x_{n+1} = x_n + \frac{h}{2}(3f_n - f_{n-1})$ - Adam Bashforth de ord. 2

e nev. de 2 valori inițiale (calculăm x_1 din x_0 cu Euler/RK2/RK4)

$K=3 \Rightarrow x_{n+1} = x_n + h\left(\frac{23}{12} f_n - \frac{4}{3} f_{n-1} + \frac{5}{12} f_{n-2}\right)$

$K=4 \Rightarrow x_{n+1} = x_n + h\left(\frac{55}{24} f_n - \frac{59}{24} f_{n-1} + \frac{37}{24} f_{n-2} - \frac{3}{8} f_{n-3}\right)$

Concluzii: - Simple, nr. minim de evaluări ale lui $f \Rightarrow$ rapidă

- folosim o interpolare pt. a aproxima pct. din afara intervalului $\Rightarrow$ erori

Metode implicite (Adam-Bashforth-Moulton)

Evităm extrapolare $\Rightarrow$ eroare mai mică

În loc de $P_K(t)$ construim $P_{K+1}(t)$ care interpolează

$[(t_{n-K}, f_{n-K}) \ldots (t_n, f_n), (t_{n+1}, f_{n+1})]$

$x_{n+1} = x_n + h(b_{-1} \underbrace{f_{n+1}}_{= f(t_{n+1}, x_{n+1})} + b_0 f_n + \ldots + b_K f_{n-K})$

implicit — explicit

Aproximăm x_{n+1} cu met. tangentei / secantei / AB2 $[x_{n-1}, x_n]$ + met. tangentei

Exemple:

$K=-1$: $x_{n+1} = x_n + h f_{n+1}$ (met. Euler implicită înapoi)

$K=0$: $x_{n+1} = x_n + \frac{h}{2}(f_n + f_{n+1})$ (reg. trapezelor)

$K=1$: $x_{n+1} = x_n + h\left(\frac{5}{12} f_{n+1} + \frac{2}{3} f_n - \frac{1}{12} f_{n-1}\right)$

Metodele multipas (ABM, Predictor-Corector)

Combină AB explicit cu ABM implicit.

1) având x_n, x_{n-1} și f_n, f_{n-1}

Predictor $\begin{cases} x^*_{n+1} = x_n + h\left(\frac{3}{2} f_n - \frac{1}{2} f_{n-1}\right) \\ f^*_{n+1} = f(t_{n+1}, x^*_{n+1}) \end{cases}$ [AB2 explicit]

2) Un pas de ABM implicit cu $K=0$, folosind f^*_{n+1}

Corector $\begin{cases} x_{n+1} = x_n + \frac{h}{2}(f^*_{n+1} + f_n) \\ f_{n+1} = f(t_{n+1}, x_{n+1}) \end{cases}$ [ABM implicit]